

Introduction to Kafka and Spark Streaming

Master M2 – Université Grenoble Alpes & Grenoble INP

2026

This lab is an introduction to Kafka and Spark Streaming. The lab assumes that you run on a Linux machine similar to the ones available in the lab rooms of Ensimag. Some information about accessing remotely to the machines of Ensimag is provided in appendix.

1 Apache Kafka

In this first part, we are going to execute basic operations with the Kafka message broker to understand how it works.

The documentation on Kafka is available here: <https://kafka.apache.org/>. It can be a good idea to refresh your mind about the description that was made of Kafka during the lectures by having a look at the corresponding slides.

Note that this first part of the lab is strongly inspired from the Kafka documentation. You can refer to it when some explanations are unclear: <https://kafka.apache.org/quickstart>

1.1 Running a Kafka broker

Note that older versions of Kafka use a ZooKeeper cluster to manage failures and broker groups. When consulting online documentation be sure to check on *Kraft* deployments of Kafka.

Kafka can be run using local scripts and downloaded files or the docker image.

1.1.1 Using a native Linux Installation

Follow the steps given at

<https://kafka.apache.org/quickstart/#step-1-get-kafka>.

1.1.2 Using a Docker Container

To run and use the Kafka docker container, follow

<https://docs.docker.com/guides/kafka/>.

When using the container, all commands from the Linux-native environment that start with `bin/some_script` become
`docker exec -ti <the name of the docker container> /opt/kafka/bin/somescript.`

1.2 First operations with Kafka

emphIn the following, the commands are given in the syntax of the native Linux version. If you are using Docker, be sure to execute the provided scripts inside the containers as indicated in the section above.

Messages in Kafka are published in a topic. The following command can be used to obtain the list of topics that already exist in a Kafka cluster:

```
> bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

At the beginning, you should observe that no topics exist. Hence we are going to create one (replace `[MY_TOPIC_NAME]` with the name you like):

```
> bin/kafka-topics.sh --create --bootstrap-server localhost:9092 \
    --replication-factor 1 --partitions 1 \
    --topic [MY_TOPIC_NAME]
```

You can then verify that this new topic exists.

It is now time to publish the first messages in the topic. To publish messages, we are going to use the Kafka client console. To start the console, run:

```
> bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
    --topic [MY_TOPIC_NAME]
```

From this point on, you can start publishing messages on the topic by entering messages in the console. However, the setup is not yet very interesting as there are no processes reading the messages published on the topic.

To read the messages published on the topic, we are going to start a consumer console:

```
> bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
    --topic [MY_TOPIC_NAME] --from-beginning
```

Note that since we used the flag `-from-beginning`, the consumer receives all the messages published since the topic has been created. To create a client that only receives messages published after it connected, simply remove this flag.

1.3 Understanding the semantics of operations with Kafka

We list below a set of questions that you should try to answer by running new tests with Kafka.

- What happens if the Kafka server crashes and needs to be restarted? Is it possible for clients that connect after the server has restarted to retrieve the messages that were published before the crash? (Try to explain)
 - To simulate a crash of the server, we are simply going to send a signal to the server process by typing `CTRL+C` in the corresponding terminal (i.e., sending a `SIGINT` signal to the process)¹.
- What happens if multiple consumers are registered to the same topic? When a message is published, which consumer receives the messages?
- What happens if multiple producers produce messages on the same topic? What is received by the consumers?
- During the lecture on Kafka, we introduced the notion of consumer group. To associate a consumer to a specific consumer group, the following comment should be used:

```
> bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
    --topic [MY_TOPIC_NAME] \
    --from-beginning \
    --group [MY_GROUP_NAME]
```

What happens on message publication when two consumers belonging to the same group are registered to the same topic? (try to explain)

To run this test, it is better to create a new topic.

- During the lecture on Kafka, we also introduced the notion of `partitions`. In the first topic we created, there was only one partition. Create a new topic that is divided into 2 partitions². Assuming that the new topic you created is called `[MY_PARTITIONED_TOPIC]`, you can verify that your new topic includes two partitions using the following command:

```
> bin/kafka-topics.sh --describe --bootstrap-server localhost:9092 \
    --topic [MY_PARTITIONED_TOPIC]
```

What happens when new messages are published on this topic if 2 consumers belonging to the same group are registered to this topic? With one producer ? With two producers ?

- Same question as before but this time with 3 clients?

1.4 Setting up a Kafka cluster

Until now we have worked with a single Kafka broker. In practice, a Kafka cluster composed of multiple Kafka brokers is deployed both to improve the performance and the reliability of the system. We are going to create our own Kafka cluster composed of 3 brokers on a single machine.

¹Students familiar with UNIX systems can try obtaining the PID of the server process using `ps aux | grep server.properties`, and can then use `kill -9` to kill the corresponding process (i.e., sending a `SIGINT` signal to the process)

²Only the value of the `-partitions` option needs to be changed in the command to create a topic

Cleaning To avoid clutter, we suggest to start this new step from a clean environment. To this end:

- Stop the existing Kafka broker by sending a `SIGINT` signal to the process.
- Delete the logs of Kafka using the following command:

```
> rm -rf /tmp/kafka-logs/* /tmp/kraft-combined-logs
```

Starting a Kafka cluster To create a Kafka cluster, we need to start multiple brokers, each with a different broker identifier. To this end, we should create 3 new configurations files, on the same model as the one we used for the first broker (file `config/server.properties`).

Before doing so, let us understand a little bit the contents of this file.

- In the beginning of the file you find the option `node.id` which is the identifier of the broker.
- Also, you will see the `process.roles` option which is set by default to `broker,controller`. The `broker` role corresponds to the message broadcasting functionality. The `controller` role is used for the servers that manage failures and decide which broker is a leader for replicated topics.
- The `listeners` option gives the ports on which the server is accepting requests. If it is a broker, it receives requests on one port, 9092 by default. If it is a controller, it receives requests on a different port, 9093 by default.
- Finally, the `log.dirs` option specifies the location for this server's logs.

To create a cluster of 3 nodes, you need to create 3 new files that are copies of the file `config/server.properties`. Let us say that these files will be `config/server-1.properties`, `config/server-2.properties` and `config/server-3.properties`.

In each new file, you will need to update (or even add) multiple options so that the values are different in each file. Namely, you will need to update:

- `node.id`: use 1, 2 and 3 in the different files.
- `controller.quorum.bootstrap.servers`: it is a good idea to list all the brokers here.
- `listeners` if you use ports 9092 and 9093 for server 1, use 9094 and 9095 for server 2, and 9096 and 9097 for server 3.
- You will need to change the values for the `advertised.listeners` option accordingly.
- You should use different directories for the `log.dirs` option in the 3 different files (e.g. `/tmp/kraft-combined-logs-1`, `/tmp/kraft-combined-logs-2` and `/tmp/kraft-combined-logs-3`)

- You should add the option `controller.quorum.voters`. As in our scenario the three servers are controllers, put `controller.quorum.voters=1@localhost:9093, 2@localhost:9095, 3@localhost:9097`

Once the new configuration files are created, you need to:

- Generate the new cluster ID (<https://kafka.apache.org/41/getting-started/quickstart/#using-downloaded-files>)

```
> KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid) "
```

- Format the log directories for the three servers (Attention, do not use the `-standalone` option)

```
> bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/server-1.properties
> bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/server-2.properties
> bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/server-3.properties
```

- Start the three brokers:

```
> bin/kafka-server-start.sh config/server-1.properties
> bin/kafka-server-start.sh config/server-2.properties
> bin/kafka-server-start.sh config/server-3.properties
```

Creating a replicated topic Once the Kafka cluster is created, new topics can be created, now with a replication degree of 3:

```
> bin/kafka-topics.sh --create --bootstrap-server localhost:9092 \
    --replication-factor 3 --partitions 1 \
    --topic [MY_REPLICATED_TOPIC]
```

You can observe the state of the new topic using the same command as introduced earlier:

```
> bin/kafka-topics.sh --describe --bootstrap-server localhost:9092 \
    --topic [MY_REPLICATED_TOPIC]
```

Some questions related to the Kafka cluster

- Explain the output of the command that describes the replicated topic
- Can a producer connect to any broker to publish a message on the topic? (To change the broker to connect to, simply change the port number in the url of the `-bootstrap-server` option)
- Create 3 consumers, each connected to a different broker. What happens when a message is published on the topic?

- Kill the broker that plays the role of leader for the replicated topic.
 - Can we still publish and consume messages on the topic?
 - Observe the new state of the replicated topic and explain what happened.
- Restart the killed broker and observe the state of the replicated topic. Explain.
- Create a new topic with a replication degree of 3 and 2 partitions. Register 3 clients belonging to the same consumer group to this topic.
 - Observe the state of the new replicated topic and explain the difference with the previous case.
 - What happens when a producer publishes messages on the topic?

2 Spark Streaming

NOTE. The following has been adapted for Spark 4. If you use other versions, you need to use compatible versions for Java and Scala. The version impacts the code of the examples, as well as the configuration files for the Scala projects, namely `build.sbt` and `build.properties`.

In this part, we are going to execute our first Spark Streaming application. The provided application is a simple version of the famous Word Count application adapted to the streaming context³. It corresponds to the tutorial example given in the documentation of Spark Structured Streaming: <https://spark.apache.org/docs/latest/streaming/getting-started.html>. The application counts the occurrences of words inside each micro-batch.

The application is also provided as an archive on the teacher's webpage (https://vmarangozova.github.io/teaching-docs/lsdm/kafka_lab/provided_code.tar.gz).

Spark Environment The following sections require that you have already Spark installed.

If this is not the case, please refer to https://vmarangozova.github.io/teaching-docs/lsdm/spark_lab/install_spark.pdf

Understanding the application Take the time to read and understand the source code of the application.

Building the application To compile the Spark Streaming application, move into the directory `code/stream_app` of the provided material and run:

```
> sbt package
```

³The original version of the code can be found in the Spark repository: <https://github.com/apache/spark/blob/v4.1.0/examples/src/main/scala/org/apache/spark/examples/streaming/NetworkWordCount.scala>

Running the application To test the application, we are going to use the network utility `Netcat`, that is going to allow us to send messages to the Spark Streaming application through the console. To start the `Netcat` server, and make it listen for new connections on port 9999, run in a dedicated terminal:

```
> nc -lk 9999
```

To start the Spark Streaming application, you can either run directly the Spark-rpvided files:

```
> [SPARK_DIRECTORY]/bin/run-example org.apache.spark.examples.streaming.NetworkWordCount
```

or use the locally compiled version

```
> [SPARK_DIRECTORY]/bin/spark-submit --master local[*] \
    ./target/scala-2.13/stream-word-count_<version>.jar localhost 9999
```

Some observations To easily observe what is happening in the Spark Streaming application, we suggest you to modify the application to increase the batch interval to 10 seconds.

After restarting your application, you can observe its state by connecting to the graphical user interface of Spark at the following url: `http://localhost:4040`. We are more specifically interested in the `Streaming` page.

You can observe that the Streaming statistics of Spark report 4 main metrics (Input rate, Scheduling delay, Processing time, Total delay):

- Observe how these metrics evolve depending on the input data.
- What information does each of these metrics provide us?

3 Kafka and Spark Streaming

Now that we have managed to have a first streaming application running, we are going to build a Spark Streaming application that reads data from a Kafka broker.

Running the provided application We provide you with another version of the streaming Word Count application, that is available in the directory `code/kafka_stream_app`:

- Compile the new application using `sbt`
- Start a Kafka broker and create a new topic
- Start the new Spark application using the following command:

```
> [SPARK_DIRECTORY]/bin/spark-submit \
    --master local[*] ./target/scala-2.13/kafka-stream-app_2.13-1.0.jar \
    localhost:9092 [CONSUMER_GROUP] [KAFKA_TOPIC]
```

Modifying the Word Count application: displaying trending words As a last step for this lab, we suggest you to modify the Word Count application to create an application that displays the most frequent words over hopping windows. To this end, you need to:

1. Aggregate the data received from the Kafka broker in overlapping windows. We want to create a new 40-second window every 10 seconds.
2. Sort the words appearing in each window in descending order according to their number of occurrence.

Most information regarding the mechanisms required to implement this application are available on this web page: <https://spark.apache.org/docs/latest/streaming-programming-guide.html#transformations-on-dstreams>.